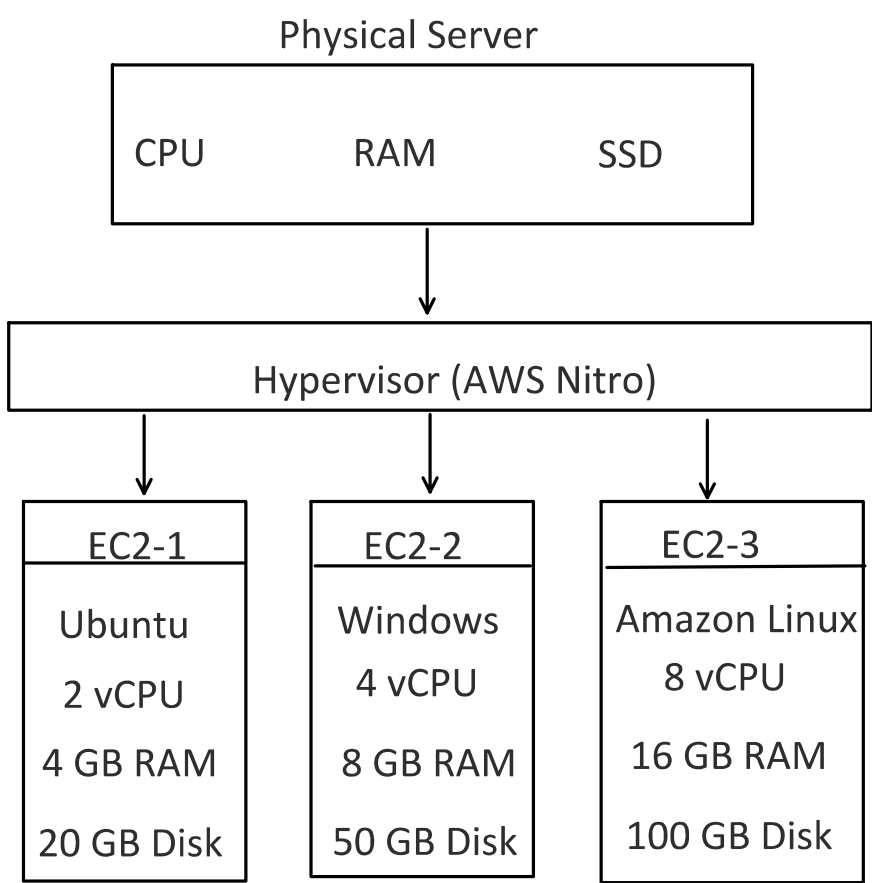


EC2 (Elastic Compute Cloud):

We can think of it as renting a virtual computer (virtual machine) in AWS data center.

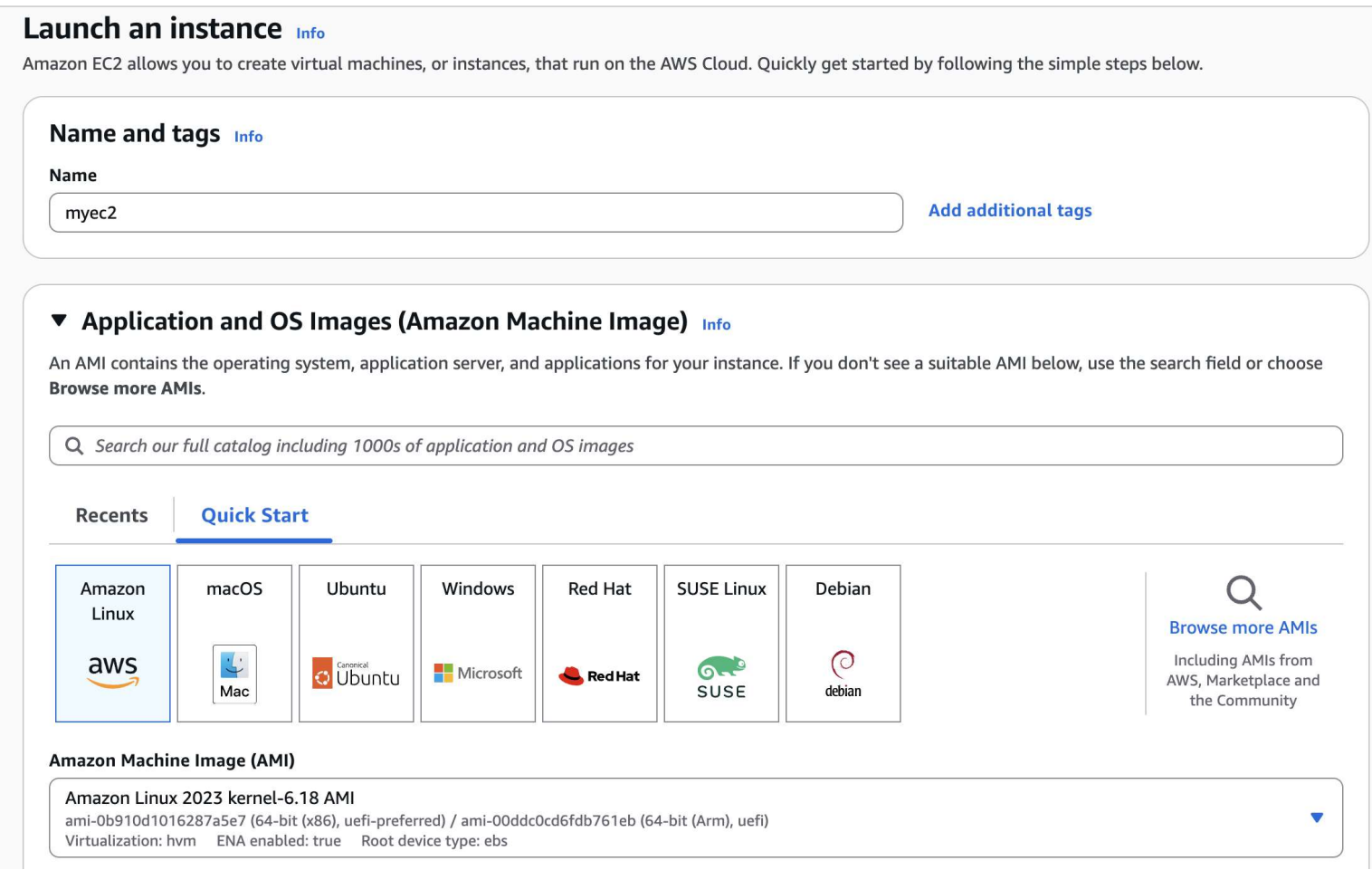
- Each EC2 behaves like a independent computer, where we can:
- o Access remotely via SSH
 - o Install any software (Java, MySQL, etc.)
 - o Run our applications 24/7
 - o Pay only for what we use



EC2 Components:

1. AMI (Amazon Machine Image)
2. Instance Type
3. Key Pair
4. Network Settings
5. EBS Volume
6. File System

AMI (Amazon Machine Image).



Pre-configured template used to launch EC2 instances.

Think of it this way:

Without an AMI, its like buying a laptop with no OS installed. Hardware exist but it has nothing to boot. So we can not use it.
So
When we launch an EC2 from an AMI, AWS creates a new virtual machine using everything stored in that AMI.

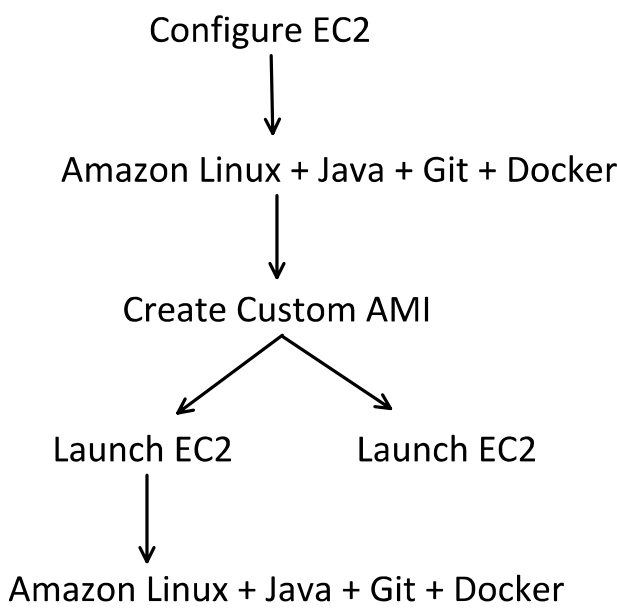
Example:
Configure the EC2 instance like we wanted:

- Create EC2:
- OS installed
 - Java installed
 - Git installed
 - Nginx installed
 - Other software's installed etc.

Then we can create AMI out of our EC2 instance.

- Now our AMI contains:
- Operating System
 - Installed packages (Java, Git, Docker, etc.)
 - System configuration
etc.

Later, whenever we launch a new EC2 using this Custom AMI, all of these are already available.



Instance Type:

▼ Instance typeInfo | Get advice

Instance type

t3.micro

Family: t3 2 vCPU 1 GiB Memory Current generation: true On-Demand Linux base pricing: 0.0112 USD per Hour On-Demand SUSE base pricing: 0.0112 USD per Hour On-Demand Ubuntu Pro base pricing: 0.0147 USD per Hour On-Demand Windows base pricing: 0.0204 USD per Hour On-Demand RHEL base pricing: 0.04 USD per Hour

Free tier eligible

☒ All generations

[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

t3.micro

Choose Server size (vCPU, RAM)

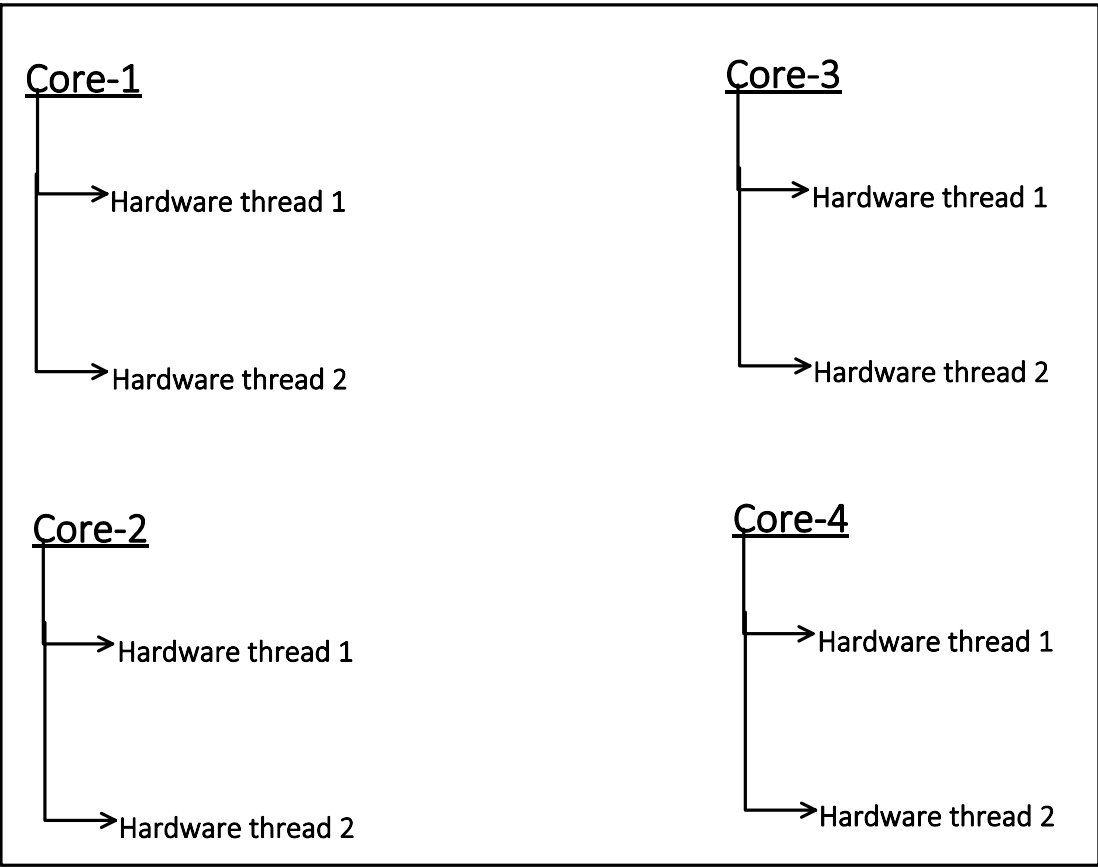
Family	Purpose	Size
t	Burstable	nano micro small medium large xlarge 2xlarge 4xlarge ...
m	General Purpose (balanced)	nano micro small medium large xlarge 2xlarge 4xlarge ...
c	Compute Optimized (more CPU)	nano micro small medium large xlarge 2xlarge 4xlarge ...
r	Memory Optimized (more RAM)	nano micro small medium large xlarge 2xlarge 4xlarge ...

Etc.

What is vCPU?

Example:

CPU (4 core)



Even though our CPU has only 4 cores, each core has 2 threads.

So total 8 vCPU.

Now when we select Instance Type say:

t3.micro -> we get 2vCPU

Means we might get:

Core-1, Thread1 = vCPU1

Core-1, thread2 = vCPU2

Key-Pair:

▼ Key pair (login)Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

my-ec2-key

Create new key pair

Create key pair

Key pair name

Key pair allow you to connect to your instance securely.

my-ec2-key

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ RSA

RSA encrypted private and public key pair

☐ ED25519

ED25519 encrypted private and public key pair

Private key file format

☒ .pem

For use with OpenSSH

☐ .ppk

For use with PuTTY

⚠ When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. Learn more

Cancel

Create key pair

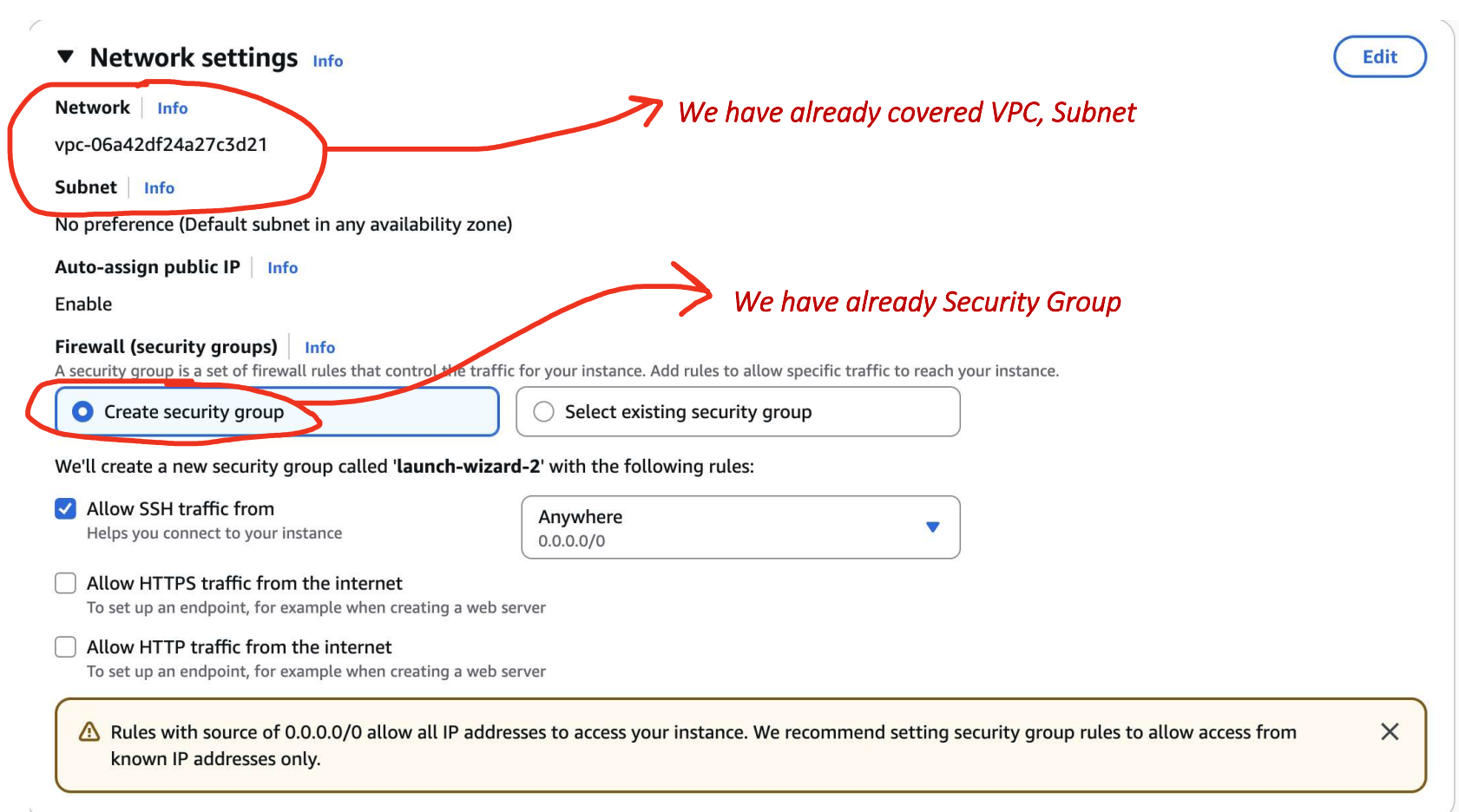
SSH Key Pair is used to securely log in to our EC2 instance without using a password.

- **Private Key (.pem):** Downloaded to our local machine.
- **Public Key:** Stored inside the EC2 instance (in the ~/.ssh/authorized_keys file).

When we try to log in:

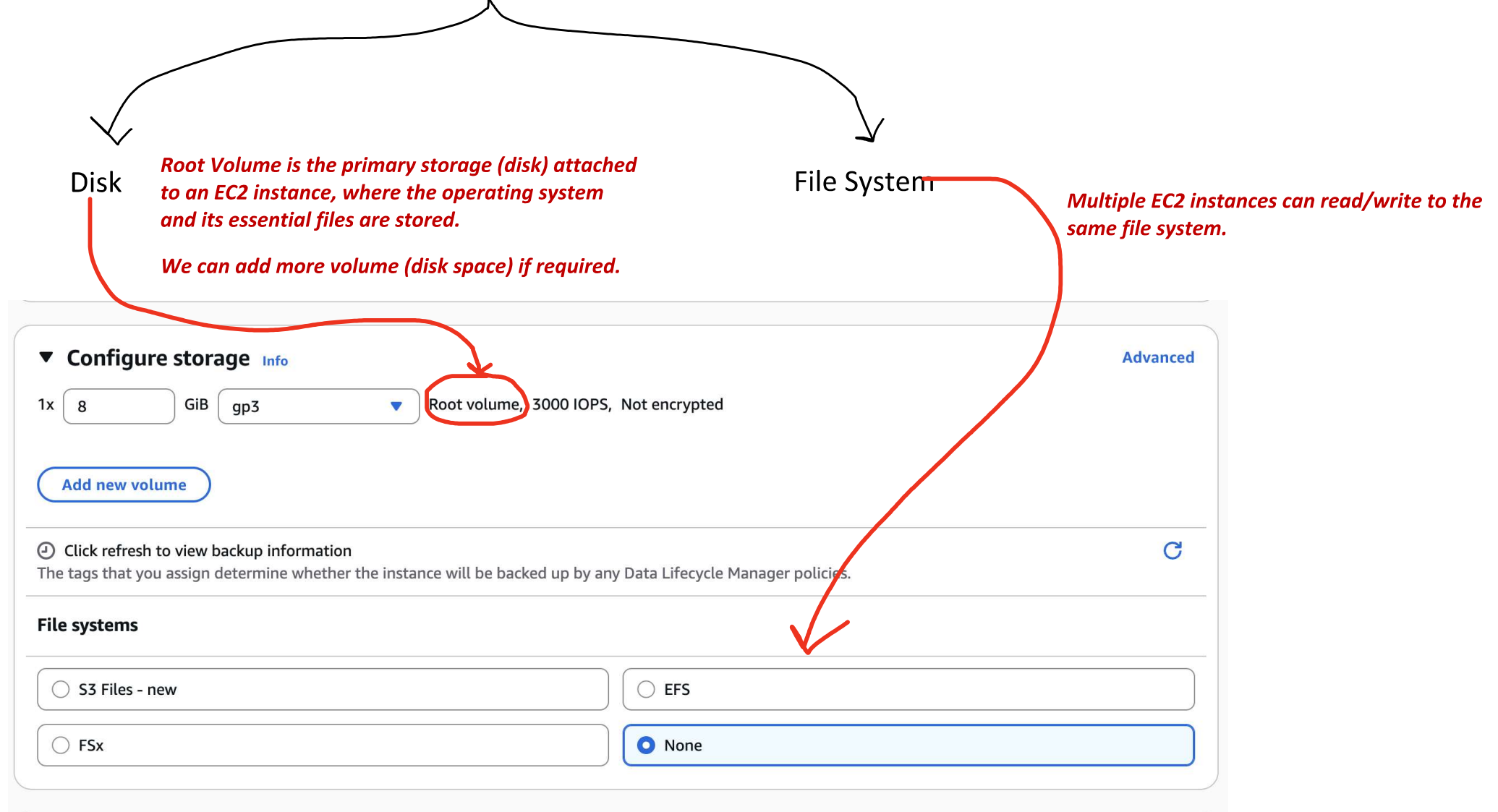
1. We provide our **private key** (generally signature is done using private key)
2. The EC2 instance already has the matching **public key**. So it will use it to verify the signature.
3. If the verification succeeds, access is granted.

Network Setting:

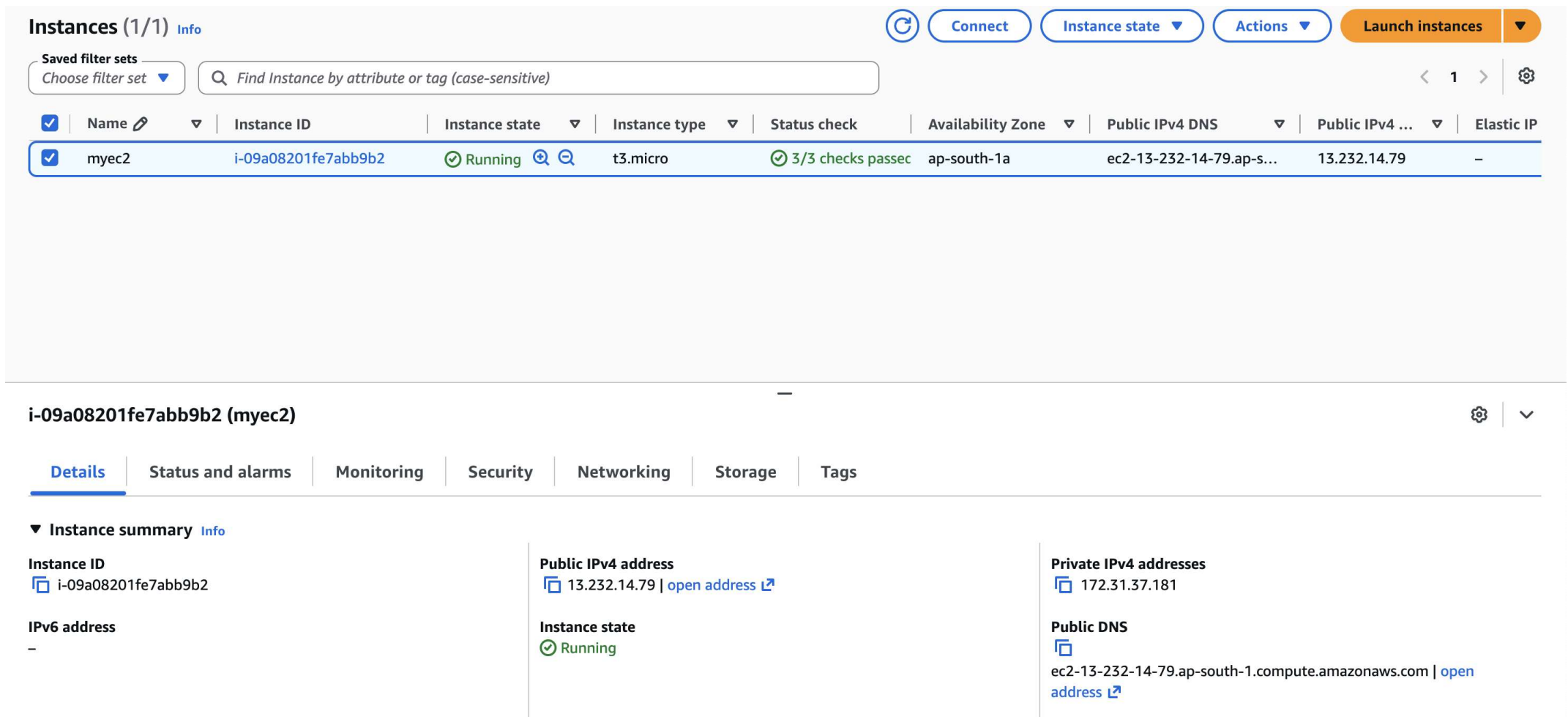


EC2 instance is created within a particular subnet (either private or public).

Configure Storage:



That's all we need to create a EC2 instance:



Now, we will install JDK, then we will deploy 1 simple springboot project

1. SSH Login

- A. Move our private key from downloads to hidden folder .ssh

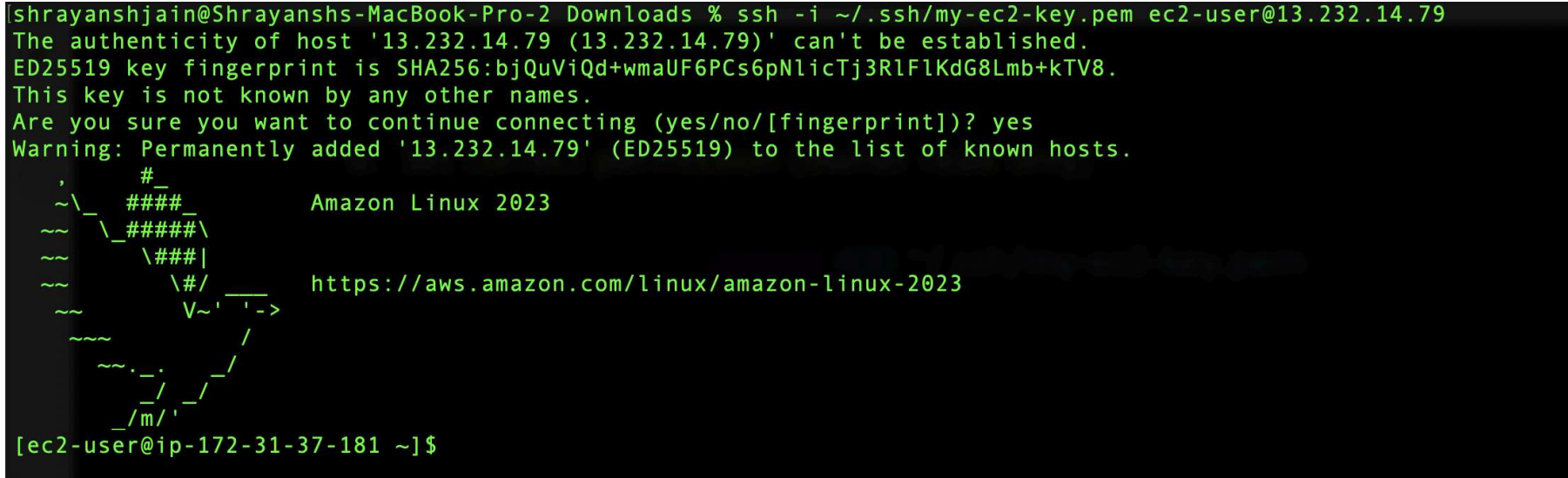
mv ~/Downloads/my-ec2-key.pem ~/.ssh/

B. Set correct permission (owner read only)

```
chmod 400 ~/.ssh/my-ec2-key.pem
```

C. Connect with EC2

```
ssh -i ~/.ssh/my-ec2-key.pem ec2-user@<<EC2 Public IP address>>
i.e
ssh -i ~/.ssh/my-ec2-key.pem ec2-user@13.232.14.79
```



D. Install Java 17

```
# Install Amazon Corretto (AWS's Java distribution)
sudo yum install -y java-17-amazon-corretto-devel

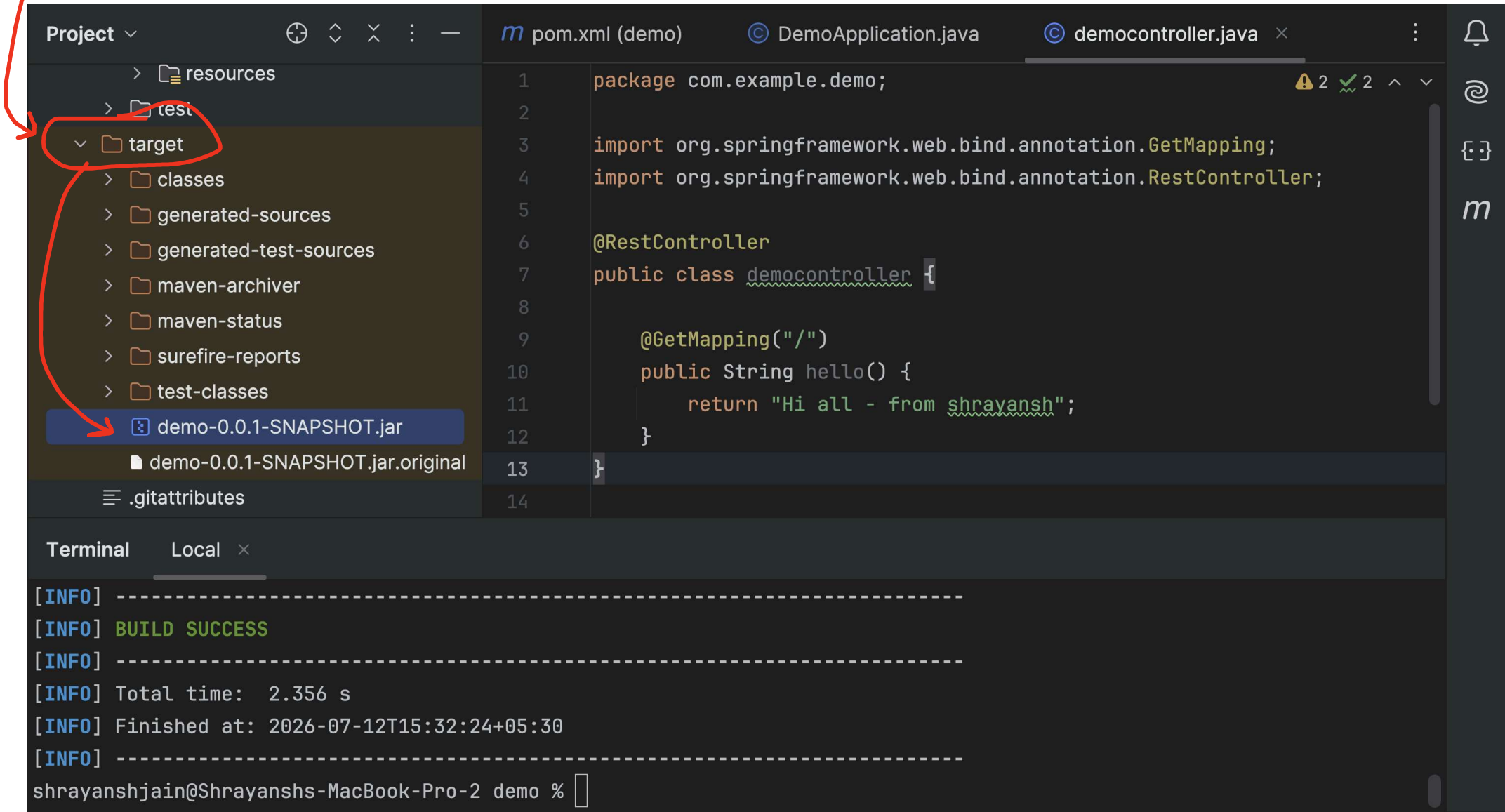
# Verify installation
java -version
```



E. Create a Jar of our SpringBoot project

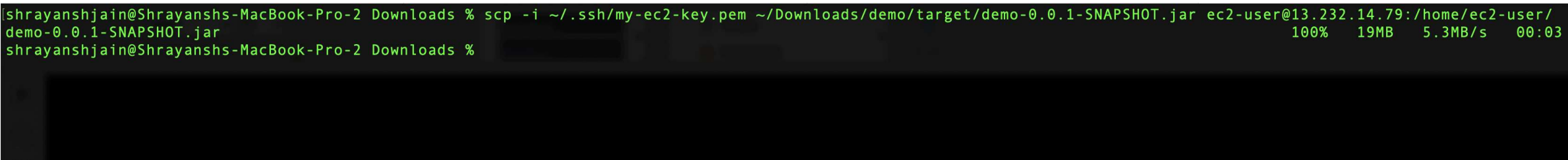
```
mvn clean package
```

Jar get created within /target folder

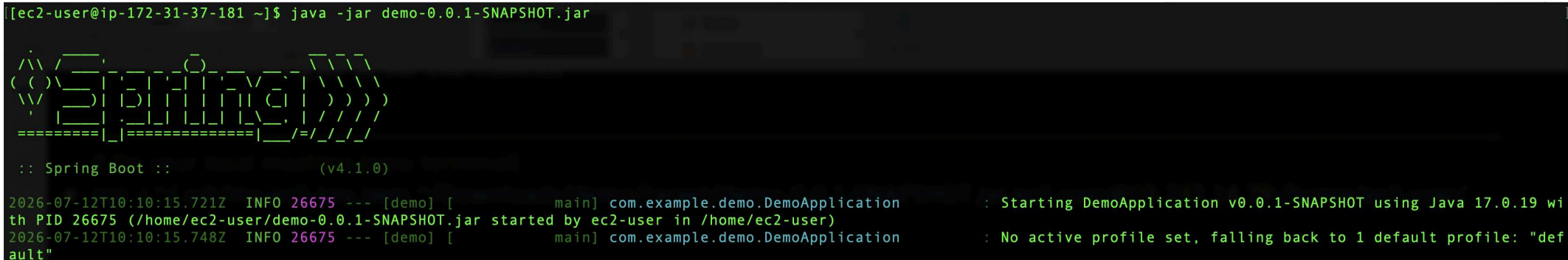


F. Upload existing jar to our EC2 instance

```
# On your local machine (new terminal)
scp -i ~/.ssh/my-ec2-key.pem ~/Downloads/demo/target/demo-0.0.1-SNAPSHOT.jar ec2-user@13.232.14.79:/home/ec2-user/
```



G. Run the Spring boot App



Lets test it:

Why? Because, we are hitting port 8080, we need to see the security group, inbound rule that 8080 is allowed or not?

Currently 8080 port is not allowed:

 Name 	Instance ID	Instance state 	Instance type 	Status check 	Availability Zone 	Public IPv4 DNS 	Public IPv4 ... 	Elastic IP 
 myec2	i-09a08201fe7abb9b2	 Running 	t3.micro	 3/3 checks passed	ap-south-1a	ec2-13-232-14-79-ap-1...	13.232.14.79	-

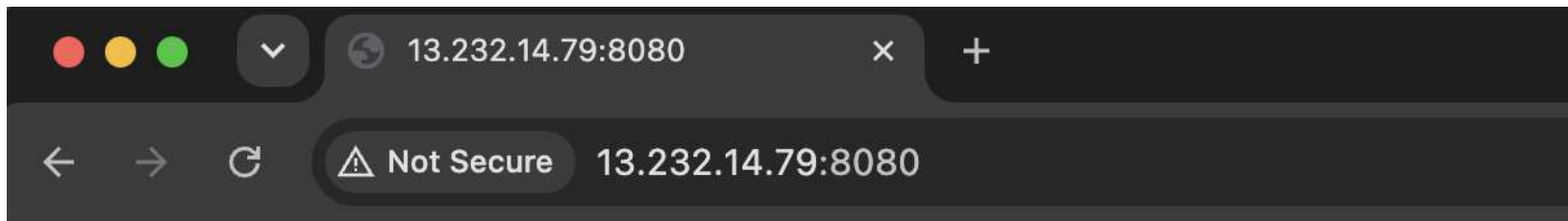
Update the security group inbound rule:

Inbound rules (3)

Manage tags
 [Edit inbound rules](#)

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
☐	-	sgr-0a47da9458754728e	IPv4	Custom TCP	TCP	8080	0.0.0.0/0
☐	-	sgr-0e9c462cc710f081c	IPv4	HTTP	TCP	80	0.0.0.0/0
☐	-	sgr-059b977c6630c83ec	IPv4	SSH	TCP	22	0.0.0.0/0

Lets test it again:



Hi all - from shrayansh

Problem 1 :

if we close the SSH, app stops.

To solve this, generally in production **systemd** service is used (daemon)

Linux boots ---> systemd starts ---> reads .services files ---> load in memory ---> wait for command service start, stop

A. Creates a config file

```
sudo nano /etc/systemd/system/myapp.service
```

B. Add the information, how to start the service

[Unit]
Description=Spring Boot Application

```
[Service]
ExecStart=/usr/bin/java -jar /home/ec2-user/demo-0.01-SNAPSHOT.jar
```

```
[Install]
WantedBy=multi-user.target
```

When to start/restart, after Linux finishes booting.

C. Save the file

D. Update the daemon, so that they know about this new service.

```
sudo systemctl daemon-reload
```

E. During first time, when Linux boots, does service need to be started? If yes, enable auto start

```
sudo systemctl enable myapp
```

F. Start the application, now linux is responsible for starting the service.

```
sudo systemctl start myapp
```

We can test by Stopping the instance and restarting it. Now we don't have to manually start the service.

Problem 2 :

If we stop the EC2 instance and Restart it. Its public IP v4 changes.

Solution: Elastic IP (static IP that doesn't change)

Steps (AWS Console):

- a. EC2 -> Elastic IPs
- b. Click "Allocate Elastic IP address"
- c. Click "Allocate"
- d. Select the new Elastic IP
- e. Actions -> Associate Elastic IP address
- f. Instance: Select your EC2
- g. Click "Associate"